

PilotFish RabbitMQ Transport – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **publisher into RabbitMQ**. The finished transaction becomes an AMQP message sent with a routing key to a queue (point-to-point) or topic exchange (publish/subscribe), with optional queue declaration and rich AMQP properties.

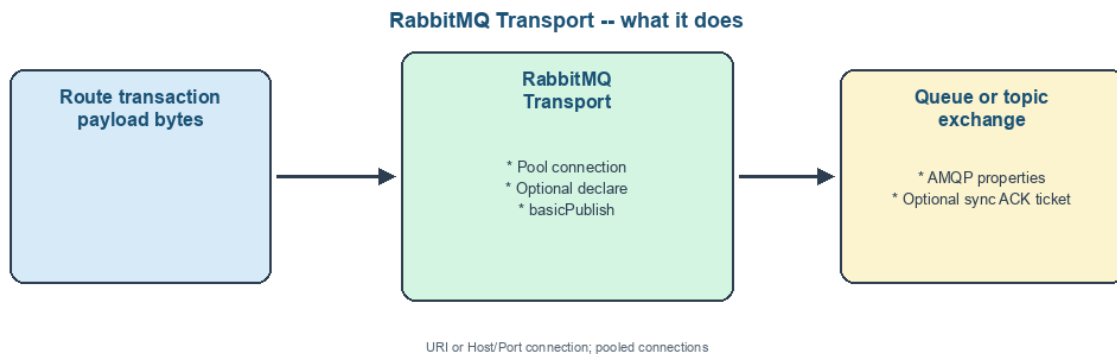


Figure 1: At a glance (plain English)

- **Connects** via URI or Host/Port list (connection pool).
- **Publishes** with routing key + exchange; Queue or Topic mode.
- **Optionally declares** queue/exchange with TTL, expire, max-length, DLX.
- **Can complete** a synchronous response ticket after publish.

Purpose

Deep dive into the **RabbitMQ** transport: connection descriptors, declare arguments, AMQP property mapping, sync-ack ticket handling, and pooling.

Provenance	Value
WAR	eip.war.hs.26R1.11
Module JAR	xcs-modules-rabbitmq-1.0.1.jar
Core JAR	xcs-core-1.2.6.jar
Decompiler	CFR 0.152
Implementation class	com.pilotfish.eip.modules.messaging.rabbitmq
Decompiled path	war-pipeline/decompiled/eip-26R1.11/.../RabbitMQ
Transport type string	RabbitMQ
Description	Publishes messages to a RabbitMQ exchange.
Help ID	console.transport.rabbitmq
Category	QUEUE / WEB

Derived from WAR bytecode (CFR). Narrative follows executable logic in `executeTransport` / `sendData`.

1. Conceptual model

```
ConnectionMethod URI | Host and Port
|
v
borrow pooled Connection -> Channel
|
v
[Declare?] queueDeclare / exchangeDeclare
|
v
build AMQP.BasicProperties (headers + props)
|
v
basicPublish(exchange, routingKey, props, bytes)
|
v
[SyncAck?] return SynchronousResponseTicket
```

2. High-level behavior

From `RabbitMQTransport.executeTransport`:

1. Read exchange, routing key (Queue tag), mode, declare flag, sync-ack flag.
2. Build `RabbitMQConnectionProperties` from URI or HostsAndPorts + user/pass/vhost/SSL.
3. `RabbitMQUtils.getConnection` (borrow from keyed pool, max 16 / key, overridable via `RABBIT_MQ_MAX_CONNECTIONS`).
4. Create channel; if Declare:
 - Queue mode + empty exchange -> `queueDeclare(routingKey, durable, exclusive, autoDelete, args)`.
 - Topic mode: null exchange becomes `amq.topic`; else `exchangeDeclare(exchange, "topic", ...)`.
5. Map Properties-tab fields into `AMQP.BasicProperties.Builder` when non-empty.
6. Merge headers from attribute map `com.pilotfish.rabbitmq.Headers` plus Headers table.
7. `basicPublish(exchange, queue, props, DataUtil.readBytes(stream))`.
8. If Sync ACK: require attribute `com.pilotfish.eip.synchronousResponseTicket` and `SynchronousResponseBroker.returnTicket`.
9. Return connection to pool (invalidate on error).
10. Legacy configs missing `ConnectionMethod` default to URI on `readConfiguration`.

3. Configuration reference

3.1 Connection (`RabbitMQUtils.getConnectionDescriptor`)

UI label	Tag	Default	Tab	Nuances
Connection Method	<code>ConnectionMethod</code>	(empty choice listed)	Connection	URI or Host and Port; required
URI	<code>URI</code>	<code>amqp://localhost</code>	Connection	When method=URI
Host and Port	<code>HostsAndPorts</code>	<code>localhost:5672</code>	Connection	List; when Host and Port
Virtual Host	<code>VirtualHost</code>	/	Connection	Host/Port mode
Username	<code>Username</code>	""	Connection	Host/Port mode
Password	<code>Password</code>	""	Connection	Host/Port mode
Use SSL?	<code>SSLEnabled</code>	false	Connection	Host/Port mode; <code>factory.useSslProtocol()</code>

3.2 Publish tab

UI label	Tag	Default	Required	Nuances
Routing Key	<code>Queue</code>	""	Yes	Used as routing key and queue name when declaring in Queue mode

UI label	Tag	Default	Required	Nuances
Exchange	Exchange	""	No	Empty + Queue mode -> declare queue named as routing key
Mode	PointToPointMode	Queue (Point-to-Point)	No	Or Topic (Publish/Subscribe)

3.3 Properties tab (AMQP BasicProperties)

UI label	Tag	Default
ContentEncoding		com.pilotfish.rabbitmq.ContentEncoding
ContentType		com.pilotfish.rabbitmq.ContentType
CorrelationId		com.pilotfish.rabbitmq.CorrelationId
DeliveryMode		com.pilotfish.rabbitmq.DeliveryMode
Expiration		com.pilotfish.rabbitmq.Expiration
MessageId		com.pilotfish.rabbitmq.MessageId
Priority		com.pilotfish.rabbitmq.Priority
ReplyTo		com.pilotfish.rabbitmq.ReplyTo
Timestamp		com.pilotfish.rabbitmq.Timestamp java.sql.Date.valueOf()
Type		com.pilotfish.rabbitmq.Type
UserId		com.pilotfish.rabbitmq.UserId
AppId		com.pilotfish.rabbitmq.AppId
Headers		com.pilotfish.rabbitmq.Headers

3.4 Advanced declare / sync

UI label	Tag	Default	Nuances
Synchronous Acknowledge	SyncAck	false	Completes sync ticket after publish
Declare	Declare	true	Gates durable/exclusive/auto-delete/TTL/expire/max/DLX fields
Durable	Durable	false	Survive broker restart
Exclusive	Exclusive	false	Queue exclusive to connection
Auto-Delete	AutoDelete	false	Delete when unused
Message TTL	MessageTTL	0	x-message-ttl when > 0 (ms)
Auto Expire	AutoExpire	0	x-expires when > 0 (ms)

UI label	Tag	Default	Nuances
Max Length	MaxLength	0	x-max-length when > 0; tooltip text incorrectly describes expire
Dead Letter Exchange	DeadLetterExchange		x-dead-letter-exchange

Inherited AbstractTransport lifecycle

From `AbstractTransport.sendData`:

1. `checkRequiredData(data)`
2. `executeTransport(data)`
3. On any exception: `setTransactionStateError()` then wrap as `EIPEException("Error executing transport", e)`
4. On success: `setTransactionStateComplete()`

`supportsBatchSynchronization()` defaults to `false`. Redis (EvalXML) overrides `sendData` and does not re-wrap through the `AbstractTransport` path in the same way (see Redis section).

4. Runtime algorithms

Queue vs Topic declare

- Queue mode only declares when exchange string is exactly "".
- Topic mode: if exchange is null -> force `amq.topic` **without declare**; if non-null -> `exchangeDeclare(..., "topic", durable, autoDelete, exclusive, arguments)`.

Sync ACK

Requires prior listener/route setup that stores `SynchronousResponseTicket` on the transaction. Missing/invalid ticket throws `EIPEException` after publish attempt path.

Connection pool

`GenericKeyedObjectPool` keyed by `RabbitMQConnectionProperties`, default max 16 per key.

5. Transaction attributes

Consumed / related

Attribute	Usage
<code>com.pilotfish.rabbitmq.Headers</code>	Optional Map merged into AMQP headers

Attribute	Usage
<code>com.pilotfish.eip.synchronousResponseTicket</code>	Required when SyncAck enabled

Config tags under `com.pilotfish.rabbitmq.*` set outbound AMQP properties (not necessarily transaction attributes).

6. Worked example

Mode Queue, Routing Key `claims.in`, Exchange blank, Declare true, Durable true:

Declares durable queue `claims.in` and publishes payload with that routing key on default exchange semantics (`basicPublish("", queue, ...)` when exchange empty).

7. Known quirks and caveats

1. **UserId bug** – decompiled call uses property name `"com.pilotfish.rabbitmq.UserId, txData"` (comma included) – `UserId` config may never apply.
2. **Max Length tooltip** copies Auto Expire wording (wrong).
3. Inner channel errors are logged and connection invalidated but **may not rethrow** before outer catch depending on control flow – verify error propagation in tests.
4. Timestamp expects `yyyy-mm-dd` (`Date.valueOf`), not full AMQP timestamps.
5. DeliveryMode / Priority parsed via `getIntValue` when non-empty strings.
6. Legacy interfaces without `ConnectionMethod` get URI forced on read.

8. Operational recommendations

Goal	Guidance
Simple queue drop	Queue mode, empty Exchange, Declare true
Fan-out topic	Topic mode + named topic exchange
High availability	Host and Port list of mirror nodes
Sync HTTP listener pairing	SyncAck + SynchronousResponseTicket on tx
Pool sizing	<code>-DRABBIT_MQ_MAX_CONNECTIONS=N</code>

9. Source index (this WAR)

Topic	Path under war-pipeline/decompiled/eip-26R1.11/
RabbitMQ transport	com/pilotfish/eip/modules/messaging/rabbitmq/RabbitMQTransport.java
Connection utils	com/pilotfish/eip/modules/messaging/rabbitmq/RabbitMQUtils.java
Connection props	com/pilotfish/eip/modules/messaging/rabbitmq/RabbitMQConnectionProperties.java
Module JAR	war-pipeline/jars/eip-26R1.11/xcs-modules-rabbitmq-1.0.1.jar

10. Pipeline provenance

PilotFish WARs/eip.war.hs.26R1.11

-> xcs-modules-rabbitmq-1.0.1.jar

-> CFR decompile

-> Documents/Transports/26R1.11/PilotFish-RabbitMQ-Transport-Reference-26R1.11.(md|pdf)

Deep-dive documentation from WAR decompile – RabbitMQ Transport – 26R1.11 – August 2026.